

TP – Programmation en PostgreSQL

Table des matières

1) Problématique.....	1
2) Prérequis	1
3) Exercices	2
4) Conclusion.....	10

1) Problématique

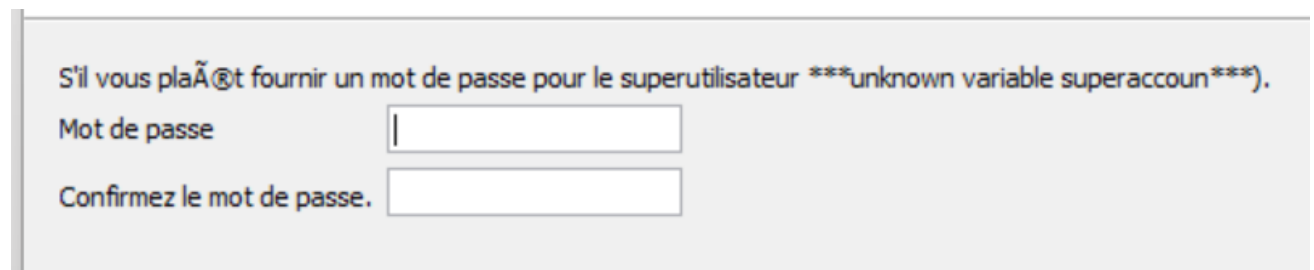
Nous souhaitons apprendre le SQL avec une autre variante de MySQL : PostgreSQL. Nous allons donc répondre à différents exercices en rapport avec cette variante en nous aidant du dossier 3.3 qui nous est fourni.

2) Prérequis

Nous allons devoir installer PostgreSQL pour effectuer les différents exemples du dossier. Pour installer PostgreSQL, que ce soit sur Windows ou Linux, il suffit simplement de se diriger vers le lien suivant afin de choisir la version qui nous intéresse :

<https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>

On nous demande par la suite, lors de l'installation, d'indiquer un mot de passe pour notre utilisateur :



S'il vous plaît fournir un mot de passe pour le superutilisateur `postgres`.

Mot de passe

Confirmez le mot de passe.

Dans mon cas je vais utiliser comme mot de passe : **init01**

Pour le reste on laisse toutes les options par défaut.

A la fin de l'installation, le logiciel StackBuilder s'ouvre, pas besoin de l'utiliser vous pouvez fermer la page.

Ensuite on appuie sur la touche Windows pour chercher « psql » afin de tomber sur le terminal de PostgreSQL. Il faudra cliquer 4 fois sur entrer pour laisser les options par défaut. Ensuite il faudra taper le mot de passe que vous avez choisi, pour moi ça sera donc ?init01

```
SQL Shell (psql)
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Mot de passe pour l'utilisateur postgres :

psql (18.6)
Attention : l'encodage console (850) diffère de l'encodage Windows (1252).
            Les caractères 8 bits peuvent ne pas fonctionner correctement.
            Voir la section « Notes aux utilisateurs de Windows » de la page
            référence de psql pour les détails.
Saisissez « help » pour l'aide.
```

Pour bien commencer, on va créer notre base de données bac à sable, où l'on y effectuera les différents exemples du dossier.

On utilise donc la commande **CREATE DATABASE nom_bdd** pour créer la base de données et la commande **\c nom_bdd**. Dans mon cas la BDD s'appellera pgsq :

```
postgres=# CREATE DATABASE pgsq;
CREATE DATABASE
postgres=# \c pgsq
Vous êtes maintenant connecté à la base de données « pgsq » en tant qu'utilisateur « postgres ».
pgsq=# |
```

Le terminal est désormais prêt à l'emploi et nous pouvons commencer à effectuer les exercices.

3) Exercices

Exercice 1 : Fonction calculer_longueur_max

On s'aide du dossier fourni pour répondre à cette question, on ajoute les deux paramètres de chaînes : mot1 et mot 2, tous deux VARCHAR. On veut un résultat sous forme d'entier. Ensuite on déclare deux variables longueur, puis on attribue la longueur de chaque mot à un nombre (résultant de la

fonction length(...). Puis on vérifie avec un IF THEN si la longueur du premier mot est plus longue que la longueur du deuxième mot, si c'est le cas alors on affiche la longueur du premier mot, sinon du deuxième.

```
pgsql=# CREATE OR REPLACE FUNCTION calculer_longueur_max(mot1 varchar, mot2 varchar)
pgsql=# RETURNS integer AS $$
pgsql$# DECLARE
pgsql$#     longueur integer;
pgsql$#     longueur2 integer;
pgsql$# BEGIN
pgsql$#     longueur := length(mot1);
pgsql$#     longueur2 := length(mot2);
pgsql$#     IF longueur >= longueur2 THEN
pgsql$#         RETURN longueur;
pgsql$#     ELSE
pgsql$#         RETURN longueur2;
pgsql$#     END IF;
pgsql$# END;
pgsql$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
pgsql=# SELECT calculer_longueur_max('bonjour', 'salut');
calculer_longueur_max
-----
                        7
(1 ligne)
```

Exercice 2 : Nb. D'occurrences d'un caractère dans un intervalle

1. Boucle FOR

On déclare dans la fonction nb_occurences_for 4 paramètres : caractere de type CHAR, cdc de type VARCHAR, debut de type integer et fin de type integer. La fonction retourne u nombre d'occurrence soit un entier donc integer.

On déclare un compteur et i qui représente l'indice. On indique que pour chaque indice de l'intervalle du début à l'intervalle de fin choisi par l'utilisateur, on souhaite en boucle jusqu'à que l'on ait parcouru le dernier indice d'extraire chaque lettre grâce à la fonction **substr** dans lequel on choisit la chaine à analyser, la variable d'indice et la taille de la part que l'on veut couper (ici 1 pour chaque indice). Après on compare la lettre extraite avec le caractère choisi par l'utilisateur et si c'est le même ça augmente de

un le compteur. Puis la fonction retourne la valeur du compteur après la boucle

```
pgsql=# CREATE OR REPLACE FUNCTION nb_occurrences_for(caractere char, cdc varchar, debut integer, fin integer) RETURNS integer AS $$
pgsql$# DECLARE
pgsql$#     compteur integer := 0;
pgsql$#     i integer;
pgsql$# BEGIN
pgsql$#     FOR i IN debut..fin LOOP
pgsql$#         IF substr(cdc, i, 1) = caractere THEN
pgsql$#             compteur := compteur + 1;
pgsql$#         END IF;
pgsql$#     END LOOP;
pgsql$#     RETURN compteur;
pgsql$# END;
pgsql$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
pgsql=# SELECT nb_occurrences_for('n', 'banane', 1, 5);
 nb_occurrences_for
-----
                2
(1 ligne)
```

2. Boucle LOOP

Ici on va utiliser LOOP, on commence par déclarer le compteur et la variable d'indice, puis on donne la valeur de début d'intervalle à l'indice. On utilise le LOOP et on donne la condition de sortie de la boucle qui lorsque i dépasse l'autre bout de l'intervalle (la fin). Ensuite on réutilise le système d'extraction d'avant mais cette fois-ci on doit manuellement indiquer à i de s'incrémenter. Puis comme avant on retourne le résultat du compteur après la boucle.

```
CREATE OR REPLACE FUNCTION nb_occurrences_loop(caractere char, cdc varchar, debut integer, fin integer) RETURNS integer AS $$
DECLARE
    compteur integer := 0;
    i integer;
BEGIN
    i := debut;

    LOOP
        EXIT WHEN i > fin;

        IF substr(cdc, i, 1) = caractere THEN
            compteur := compteur + 1;
        END IF;

        i := i + 1;
    END LOOP;

    RETURN compteur;
END;
$$ LANGUAGE plpgsql;
FUNCTION
```

Résultat :

```
pgsql=# SELECT nb_occurrences_loop('n', 'banane', 1, 5);
nb_occurrences_loop
-----
2
(1 ligne)
```

3. Boucle WHILE

Ici c'est WHILE, on fait comme avant, on attribue la valeur d'intervalle du début à i mais cette fois-ci on utilise WHILE i inférieur ou égal à l'intervalle de fin au lieu de le faire en condition de sortie à l'intérieur de la boucle. Toujours la même fonction d'extraction et comme pour le loop on doit ajouter l'incrément de i.

```
pgsql=# CREATE OR REPLACE FUNCTION nb_occurrences_while(caractere char, cdc varchar, debut integer, fin integer) RETURNS integer AS $$
pgsql$# DECLARE
pgsql$#     compteur integer := 0;
pgsql$#     i integer;
pgsql$# BEGIN
pgsql$#     i := debut;
pgsql$#     WHILE i <= fin LOOP
pgsql$#         IF substr(cdc, i, 1) = caractere THEN
pgsql$#             compteur := compteur + 1;
pgsql$#         END IF;
pgsql$#         i := i + 1;
pgsql$#     END LOOP;
pgsql$#     RETURN compteur;
pgsql$# END;
pgsql$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
pgsql=# SELECT nb_occurrences_while('n', 'banane', 1, 5);
nb_occurrences_while
-----
2
(1 ligne)
```

Exercice 3 : Nombre de jours du mois selon date donné

Ici nous allons demander à l'utilisateur d'insérer une date en paramètre pour la fonction, on déclare une variable m (mois) de type texte et nb_jours de type entier.

Ensuite on utilise la fonction `to_char(d, 'MM')` afin de convertir seulement la partie MM de la date en texte pour ensuite la stocker dans m.

Ensuite on vérifie pour chaque cas combien vaut `nb_jours`, si m est égal à l'un des mois de 31 jours alors `nb_jours` sera égal à 31, pareil pour les mois de 30 jours et février. Ensuite on retourne à l'utilisateur le nombre de jours par rapport au test précédent.

```
pgsql=# CREATE OR REPLACE FUNCTION getNbJoursParMois(d date) RETURNS integer AS $$
pgsql$# DECLARE
pgsql$#     m text;
pgsql$#     nb_jours integer;
pgsql$# BEGIN
pgsql$#     m := to_char(d, 'MM');
pgsql$#     CASE m
pgsql$#         WHEN '01', '03', '05', '07', '08', '10', '12' THEN
pgsql$#             nb_jours := 31;
pgsql$#         WHEN '04', '06', '09', '11' THEN
pgsql$#             nb_jours := 30;
pgsql$#         WHEN '02' THEN
pgsql$#             nb_jours := 28;
pgsql$#     END CASE;
pgsql$#     RETURN nb_jours;
pgsql$# END;
pgsql$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
pgsql=# SELECT getNbJoursParMois('28/04/2007');
getnbjoursparmois
-----
                30
(1 ligne)
```

Exercice 4 : Format SQL au format JJ/MM/AA

On déclare la fonction qui va retourner sous forme de texte (VARCHAR).

On déclare une variable de date de type varchar : `date_fr`. Cette variable va récupérer encore une fois grâce à la fonction vu précédemment (elle est capable de reconnaître DD, MM et YY et grâce à ça on peut facilement réarranger en lui donnant un ordre précis) puis on retourne la date.

```
pgsql=# CREATE OR REPLACE FUNCTION dateSqlToDatefr(d date) RETURNS varchar AS $$
pgsql$# DECLARE
pgsql$#     date_fr varchar;
pgsql$# BEGIN
pgsql$#     date_fr := to_char(d, 'DD/MM/YY');
pgsql$#     RETURN date_fr;
pgsql$# END;
pgsql$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
ASTUCE : Aucune fonction ne correspond au nom donné et aux types d'arguments.
Vous devez ajouter des conversions explicites de type.
pgsql=# SELECT dateSqlToDatefr('2026-09-11');
      dateSqlToDatefr
-----
11/09/26
(1 ligne)
```

Exercice 5 : Indiquer le nom du jour de la date

On commence par déclarer une variable pour le numéro du jour et une pour le nom. Puis on attribue le Day of the Week (DOW) qui représente les indices 0 à 6 : Dimanche à Samedi. Par rapport à cette connaissance on peut donc utiliser un CASE qui permettra d'attribuer un nom selon le num résultant de DOW et de stocker le nom dans la variable nom_jour pour que l'on puisse à la fin le retourner.

```
pgsql=# CREATE OR REPLACE FUNCTION getNomJour(d date) RETURNS varchar AS $$
pgsql$# DECLARE
pgsql$#     num_jour integer;
pgsql$#     nom_jour varchar;
pgsql$# BEGIN
pgsql$#     num_jour := extract(DOW FROM d);
pgsql$#
pgsql$#     CASE num_jour
pgsql$#         WHEN 0 THEN nom_jour := 'Dimanche';
pgsql$#         WHEN 1 THEN nom_jour := 'Lundi';
pgsql$#         WHEN 2 THEN nom_jour := 'Mardi';
pgsql$#         WHEN 3 THEN nom_jour := 'Mercredi';
pgsql$#         WHEN 4 THEN nom_jour := 'Jeudi';
pgsql$#         WHEN 5 THEN nom_jour := 'Vendredi';
pgsql$#         WHEN 6 THEN nom_jour := 'Samedi';
pgsql$#     END CASE;
pgsql$#
pgsql$#     RETURN nom_jour;
pgsql$# END;
pgsql$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
pgsql=# SELECT getNomJour('2007-04-28');
      getnomjour
-----
Samedi
(1 ligne)
```

Exercice 6 :

Ici nous commençons d'abord par nous connecter à la base de données fournie sur le site. On utilise \i + l'emplacement du fichier pour que le script créé automatiquement les requêtes nécessaires et nous connecte à la base :

```
postgres=# \i 'C:/Users/Ewan Alexandre/Desktop/postgresql/hhb_direct_v1 (1).sql'
SET
SET
SET
SET
SET
SET
SET
CREATE DATABASE
ALTER DATABASE
Vous êtes maintenant connecté à la base de données « hhbdirect » en tant qu'utilisateur « postgres ».
SET
SET
SET
SET
SET
psql:C:/Users/Ewan Alexandre/Desktop/postgresql/hhb_direct_v1 (1).sql:38: ERREUR:  l'extension « plpgsql » existe déjà
A
ALTER LANGUAGE
```

Maintenant on va créer une fonction qui déclare une variable nb qui va stocker le nombre de client débiteurs puis on l'affiche avec le RETURN (on doit vérifier d'abord si le solde est inférieur à 0) :

```
hhbdirect=# CREATE OR REPLACE FUNCTION nb_clients_debiteurs()
hhbdirect-# RETURNS integer AS $$
hhbdirect-# DECLARE
hhbdirect-#     nb integer;
hhbdirect-# BEGIN
hhbdirect-#     SELECT count(DISTINCT num_client) INTO nb
hhbdirect-#     FROM posseder
hhbdirect-#     JOIN compte ON posseder.id_type = compte.id_type AND posseder.num_compte = compte.num_compte
hhbdirect-#     WHERE solde < 0;
hhbdirect-#     RETURN nb;
hhbdirect-# END;
hhbdirect-# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
hhbdirect=# SELECT nb_clients_debiteurs();
 nb_clients_debiteurs
-----
                    1
(1 ligne)
```

Ensuite, on souhaite voir le nombre de clients habitants dans une ville, on met donc la variable nom_ville en paramètres, ensuite on cherche précisément la variable nom_ville parmi les ensembles d'adresses pour ensuite retourner le nombre :

```

hhbdirect=# CREATE OR REPLACE FUNCTION nb_clients_ville(nom_ville varchar)
hhbdirect=# RETURNS integer AS $$
hhbdirect$$ DECLARE
hhbdirect$$     nb integer;
hhbdirect$$ BEGIN
hhbdirect$$     SELECT count(*) INTO nb
hhbdirect$$     FROM client
hhbdirect$$     WHERE adresse_client LIKE '%' || nom_ville || '%';
hhbdirect$$
hhbdirect$$     RETURN nb;
hhbdirect$$ END;
hhbdirect$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION

```

Résultat :

```

hhbdirect=# SELECT nb_clients_ville('LANNION');
 nb_clients_ville
-----
                2
(1 ligne)

```

Exercice 7 :

Désormais nous allons créer une fonction qui permet d'insérer un tuple dans la table client. On crée donc la fonction `insérer_client` qui va prendre nom, prenom, adresse, identifiant et mdp en paramètres puis on ajoute 1 à `max(num client)` pour le stocker dans la variable `nouveau_num` puis on ajoute le tuple `nouveau_num` puis ensuite on vérifie l'insertion, si ça marche ça nous met un message de confirmation (`true (t)`) sinon ça retourne une erreur (`false (f)`)

```

hhbdirect=# CREATE OR REPLACE FUNCTION inserer_client(
hhbdirect(#     nom varchar,
hhbdirect(#     prenom varchar,
hhbdirect(#     adresse varchar,
hhbdirect(#     identifiant varchar,
hhbdirect(#     mdp varchar
hhbdirect(# )
hhbdirect=# RETURNS boolean AS $$
hhbdirect$$ DECLARE
hhbdirect$$     nouveau_num integer;
hhbdirect$$ BEGIN
hhbdirect$$     SELECT max(num_client) + 1 INTO nouveau_num FROM client;
hhbdirect$$
hhbdirect$$     INSERT INTO client VALUES (nouveau_num, nom, prenom, adresse, identifiant, mdp);
hhbdirect$$
hhbdirect$$     RETURN true;
hhbdirect$$ EXCEPTION
hhbdirect$$     WHEN OTHERS THEN
hhbdirect$$         RETURN false;
hhbdirect$$ END;
hhbdirect$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION

```

Résultat :

```
hhbdirect=# SELECT inserer_client('MARTIN', 'Lucas', '10 Rue de Paris, 75001 PARIS', 'lmartin', 'pass123');
inserer_client
-----
t
(1 ligne)

hhbdirect=# SELECT * FROM client WHERE nom_client = 'MARTIN';
 num_client | nom_client | prenom_client |      adresse_client      | identifiant_internet | mdp_internet
-----+-----+-----+-----+-----+-----
          6 | MARTIN    | Lucas        | 10 Rue de Paris, 75001 PARIS | lmartin              | pass123
(1 ligne)
```

4) Conclusion

Pour conclure, ce TP m'a appris à utiliser la variante PostgreSQL dans différents contextes. PostgreSQL est l'un des langages SQL les plus utilisés. Ce cours pourra possiblement m'être très utile dans le futur !